Copyright 2026 Rajat Gajbhiye. Free preview. Please don't resell it.

FLIPKART · START HERE

How the Flipkart interview works

Based on first-hand candidate reports of Flipkart interviews held in 2024–26 for software engineering roles, in India where a location is given: mostly experienced backend hires (SDE-2 above all), and some campus hiring through Flipkart GRiD and SDE-1 drives. Formats vary by team and level, so treat this as the usual shape, not a fixed script.

Online test
Campus and some experienced drives: two or three DSA problems; GRiD's 2026 test added SQL and Python.

Q8

Machine coding
The first round for most SDE-2 and SDE-3 hires: about 90 minutes to build a problem such as BNPL, in memory.

Q1 Q2 Q4 Q5

Machine coding review
30 to 45 minutes on your code: its tests, its patterns, a feature added live, two requests at once.

Q1 Q2 Q4 Q5

PSDS
Usually two LeetCode medium or hard problems, sometimes as pseudocode in a shared doc.

Q7 Q8 Q9

Design
LLD and HLD: entities, schema and APIs, then scale and concurrency; booking systems came up most.

Q3 Q5 Q6

Technical
Campus interviews: DSA problems, CS basics (OOP, DBMS, operating systems) and your projects.

Q7 Q8 Q9

Techno-managerial
Your projects and why Flipkart; for a campus hire, sometimes a small design such as a booking system.

Q3

Hiring manager
Your project in depth (architecture, decisions, trade-offs), ownership, behaviour, why you're moving.

> Most experienced hires start with machine coding, and its review asks about the code you wrote: design patterns, a feature added live, errors and concurrency. Start with Questions 1 and 2.

WHERE TO START, BY EXPERIENCE

L2 2–5 years	Q1 Q2 Q3 Q4 Q5 Q7 Q8 Q9
L3 5–9 years	Q6

Main questions by level. Each ends with "What they ask next", L1 to L3 or L4: answer at your level, then one higher.

FLIPKART · ALL QUESTIONS

Every question, by round

Click a question to jump to it. Some questions were reported in more than one round and appear under each.

ONLINE TEST

Q8	Word Break: can a string be split into dictionary words? Then list every split as a...	L2	38

MACHINE CODING

Q1	Machine coding: build a food ordering system. Restaurants have menus and a...	L2	3
Q2	Machine coding: build Flipkart's Buy Now Pay Later (BNPL). Users get a credit...	L2	9
Q4	Machine coding: build a Q&A platform like Stack Overflow. Users follow topics, ask...	L2	20
Q5	Machine coding: build a Cult.fit-style gym slot booking app: centres, workout slots...	L2	25

MACHINE CODING REVIEW

Q1	Machine coding: build a food ordering system. Restaurants have menus and a...	L2	3
Q2	Machine coding: build Flipkart's Buy Now Pay Later (BNPL). Users get a credit...	L2	9
Q4	Machine coding: build a Q&A platform like Stack Overflow. Users follow topics, ask...	L2	20
Q5	Machine coding: build a Cult.fit-style gym slot booking app: centres, workout slots...	L2	25

PSDS

Q7	Topological sort: find an order to take courses with prerequisites (Course Schedule...	L2	35
Q8	Word Break: can a string be split into dictionary words? Then list every split as a...	L2	38
Q9	Count the nodes of a complete binary tree in less than O(n), then find the...	L2	41

DESIGN

Q3	Design BookMyShow's tables, with the queries for free seats, booking and booking...	L2	15
Q5	Machine coding: build a Cult.fit-style gym slot booking app: centres, workout slots...	L2	25
Q6	Design Spotify: songs that are singles or on albums, sung by one or more singers...	L3	30

TECHNICAL

Q7	Topological sort: find an order to take courses with prerequisites (Course Schedule...	L2	35
Q8	Word Break: can a string be split into dictionary words? Then list every split as a...	L2	38
Q9	Count the nodes of a complete binary tree in less than O(n), then find the...	L2	41

TECHNO-MANAGERIAL

Q3	Design BookMyShow's tables, with the queries for free seats, booking and booking...	L2	15

More Flipkart questions, with pointers and short answers, start on page 44.

FLIPKART · QUESTION 1

Machine coding: build a food ordering system. Restaurants have menus and a capacity, a selection strategy picks who makes each item, and dispatching an order frees its capacity.

Asked in Flipkart interviews (2024–26) · Frequently asked

L2 | Machine coding | Machine coding review | Backend Core | Low-level design | Core Java | Concurrency

30-SECOND ANSWER

> Model Restaurant (menu, capacity, items in progress), Order (who makes each item, at what price), a SelectionStrategy interface with a class per rule, and a FoodOrdering service that owns the rules. `place()` checks the input, asks the strategy for a restaurant per item, checks that pick, then takes room in every restaurant the order uses, or in none; a refusal is a FoodException whose reason says why (NO_ROOM, UNKNOWN_ITEM, BAD_INPUT). `dispatch()` frees an order's room once, so a repeated message changes nothing. The public methods are synchronized: choosing restaurants and taking their room is one step. Cheapest per item is greedy, and it can refuse an order another split would serve: say so before the interviewer does.

DEEP DIVE

Build the core first. The reports describe a briefing, about 90 minutes of coding, then a review where the interviewer reads, runs and extends your code, so get onboarding, menu changes, placing, dispatching and the per-restaurant counts working before any bonus. In the run, order 1 split [Idli, Pongal] between Tiffin Box and Dosa Point, order 2 filled Tiffin Box, and order 3 went to Udupi Corner. An order copies its prices when it is placed: after Tiffin Box raised Idli to 65, order 1 still totalled 80.

The service owns the rules; a strategy only chooses. `place()` first tells "never" from "not now": an item on no menu is UNKNOWN_ITEM, an order nobody has room for is NO_ROOM. The strategy gets read-only Kitchen views, which can't take room, and the service checks its pick before taking any room: every item placed once, at a restaurant that sells it, within that restaurant's free room. When a strategy sent 2 items to a kitchen with room for 1, `place()` threw IllegalStateException and the room stayed at 1; one that cast its view to Restaurant to take room got ClassCastException. That check is what lets you add strategies later without touching the service.

Greedy can refuse an order that could be served. Tiffin Box sells Idli and Vada for 30 each with room for 1; Udupi Corner sells only Idli, for 40. For [Idli, Vada], CheapestPerItem gives Idli to Tiffin Box and then has nowhere for Vada: NO_ROOM. A full search serves it, Idli from Udupi Corner and Vada from Tiffin Box, for 70. In 100,000 random orders (this data is described under the output), the greedy refused 3,068 of the 46,532 that some split could serve, and charged more than the best split on 1,911. If that matters, search the splits: an order has a few items, and 4 items over 4 restaurants is at most 256 splits (computed). FullSearch in the test is such a strategy, plugged in without changing the service.

Kitchen messages repeat. A second dispatch of the same order returns false and frees nothing; an unknown id is UNKNOWN_ORDER. Items served are counted at dispatch. Prices are ints and totals longs: three items at Integer.MAX_VALUE came to 6,442,450,941, and a capacity of Integer.MAX_VALUE works. The model keeps the problem's assumption of one of each item per order; quantities would make an order a map from item to count.

CODE

FoodOrdering.java · compiles for Java 8, tested on JDK 11, 21 and 25

```java
import java.util.*;

/** Why a request was refused. Callers branch on the reason, never on the message. */
enum Reason { BAD_INPUT, DUPLICATE, UNKNOWN_RESTAURANT, UNKNOWN_ITEM, UNKNOWN_ORDER, NO_ROOM }

class FoodException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    final Reason reason;

    FoodException(Reason reason, Object detail) {
        super(reason + ": " + detail);
        this.reason = reason;
    }
}

/** A restaurant: its menu (item -> price in rupees) and how many items it can make at once. */
class Restaurant {
    final String name;
    final int capacity;
    private Map<String, Integer> menu;
    private int inProgress;                       // items accepted, not yet dispatched
    private long served;                          // items dispatched so far

    Restaurant(String name, Map<String, Integer> menu, int capacity) {
        this.name = name;
        this.menu = menu;
        this.capacity = capacity;
    }

    Integer price(String item) { return menu.get(item); }
    int free() { return capacity - inProgress; }
    long served() { return served; }
    void setMenu(Map<String, Integer> menu) { this.menu = menu; }
    void take() { inProgress++; }
    void done() { inProgress--; served++; }

    final Kitchen view = new Kitchen() {              // what a strategy sees: read-only
        public String name() { return name; }
        public Integer price(String item) { return Restaurant.this.price(item); }
        public int free() { return Restaurant.this.free(); }
    };
}

/** An accepted order: who makes each item, at the price when it was placed. Immutable. */
class Order {
    final int id;
    final Map<String, String> from;               // item -> the restaurant making it
    final Map<String, Integer> price;             // item -> its price when ordered

    Order(int id, Map<String, Restaurant> pick) {
        Map<String, String> names = new LinkedHashMap<>();
        Map<String, Integer> prices = new LinkedHashMap<>();
        for (Map.Entry<String, Restaurant> e : pick.entrySet()) {
            names.put(e.getKey(), e.getValue().name);
            prices.put(e.getKey(), e.getValue().price(e.getKey()));
        }
        this.id = id;
        this.from = Collections.unmodifiableMap(names);
        this.price = Collections.unmodifiableMap(prices);
    }

    long total() { return price.values().stream().mapToLong(Integer::longValue).sum(); }
}
```

```java
/** A restaurant as a strategy sees it: no way to change it. */
interface Kitchen {
    String name();
    Integer price(String item);
    int free();
}

/** Picks a restaurant for every item, or returns null when the order can't be served now. */
interface SelectionStrategy {
    Map<String, Kitchen> choose(List<String> items, Collection<Kitchen> kitchens);
}

/** Each item from the cheapest restaurant with room left, counting this order's own items. */
class CheapestPerItem implements SelectionStrategy {
    public Map<String, Kitchen> choose(List<String> items, Collection<Kitchen> all) {
        Map<String, Kitchen> pick = new LinkedHashMap<>();
        Map<Kitchen, Integer> used = new HashMap<>();
        for (String item : items) {
            Kitchen best = null;
            for (Kitchen r : all) {
                Integer p = r.price(item);
                if (p == null || r.free() <= used.getOrDefault(r, 0)) continue;
                if (best == null || p < best.price(item)) best = r;   // a tie keeps the first
            }
            if (best == null) return null;
            pick.put(item, best);
            used.merge(best, 1, Integer::sum);
        }
        return pick;
    }
}

public class FoodOrdering {
    private final Map<String, Restaurant> restaurants = new LinkedHashMap<>();
    private final Map<Integer, Order> orders = new HashMap<>();
    private final Set<Integer> dispatched = new HashSet<>();
    private final SelectionStrategy strategy;
    private int nextId = 1;

    public FoodOrdering(SelectionStrategy strategy) {
        this.strategy = Objects.requireNonNull(strategy, "strategy");
    }

    public synchronized void addRestaurant(String name, Map<String, Integer> menu,
                                           int capacity) {
        if (name == null || name.trim().isEmpty() || capacity <= 0)
            throw new FoodException(Reason.BAD_INPUT, "name or capacity");
        if (restaurants.containsKey(name))
            throw new FoodException(Reason.DUPLICATE, name);
        restaurants.put(name, new Restaurant(name, checkedMenu(menu), capacity));
    }
```

```
    public synchronized void updateMenu(String name, Map<String, Integer> menu) {
        Map<String, Integer> checked = checkedMenu(menu);     // check all, then change
        restaurant(name).setMenu(checked);
    }

    /** Accepts the order only if every item can be made now; reserves room for all or none. */
    public synchronized Order place(List<String> items) {
        List<String> list = checkedItems(items);
        for (String item : list) {
            boolean sold = false;
            for (Restaurant r : restaurants.values()) sold |= r.price(item) != null;
            if (!sold) throw new FoodException(Reason.UNKNOWN_ITEM, item);
        }
        List<Kitchen> views = new ArrayList<>();
        for (Restaurant r : restaurants.values()) views.add(r.view);
        Map<String, Kitchen> pick = strategy.choose(Collections.unmodifiableList(list),
                Collections.unmodifiableList(views));
        if (pick == null)
            throw new FoodException(Reason.NO_ROOM, list.toString());
        Map<String, Restaurant> chosen = checkPick(list, pick);
        Order order = new Order(nextId++, chosen);
        for (Restaurant r : chosen.values()) r.take();
        orders.put(order.id, order);
        return order;
    }

    /** The kitchen says the order left: frees its room once; a repeat returns false. */
    public synchronized boolean dispatch(int orderId) {
        Order order = orders.get(orderId);
        if (order == null)
            throw new FoodException(Reason.UNKNOWN_ORDER, orderId);
        if (!dispatched.add(orderId)) return false;
        for (String name : order.from.values()) restaurants.get(name).done();
        return true;
    }

    public synchronized int free(String name) { return restaurant(name).free(); }
    public synchronized long served(String name) { return restaurant(name).served(); }

    // A strategy is a plug-in: it sees read-only views, and its pick is checked, so a buggy
    // one can't overfill a kitchen.
    private Map<String, Restaurant> checkPick(List<String> items, Map<String, Kitchen> pick) {
        Map<String, Restaurant> chosen = new LinkedHashMap<>();
        Map<Restaurant, Integer> used = new HashMap<>();
        for (String item : items) {
            Kitchen k = pick.get(item);
            Restaurant r = k == null ? null : restaurants.get(k.name());
            if (r == null || r.view != k || r.price(item) == null
                    || used.merge(r, 1, Integer::sum) > r.free())
                throw new IllegalStateException("strategy broke the rules for " + item);
            chosen.put(item, r);
        }
        if (pick.size() != items.size()) throw new IllegalStateException("extra items picked");
        return chosen;
    }
```

```
    private Restaurant restaurant(String name) {
        Restaurant r = name == null ? null : restaurants.get(name);
        if (r == null) throw new FoodException(Reason.UNKNOWN_RESTAURANT, name);
        return r;
    }

    private static Map<String, Integer> checkedMenu(Map<String, Integer> menu) {
        if (menu == null || menu.isEmpty())
            throw new FoodException(Reason.BAD_INPUT, "empty menu");
        Map<String, Integer> copy = new LinkedHashMap<>();
        for (Map.Entry<String, Integer> e : menu.entrySet()) {
            String item = e.getKey();
            Integer price = e.getValue();
            if (item == null || item.trim().isEmpty() || price == null || price <= 0)
                throw new FoodException(Reason.BAD_INPUT, "menu entry " + item);
            copy.put(item, price);
        }
        return Collections.unmodifiableMap(copy);
    }

    private static List<String> checkedItems(List<String> items) {
        List<String> copy = items == null ? new ArrayList<String>() : new ArrayList<>(items);
        if (copy.isEmpty() || copy.contains(null) || new HashSet<>(copy).size() != copy.size())
            throw new FoodException(Reason.BAD_INPUT, "one of each item: " + items);
        return copy;
    }
}
```

```
order 1 [Idli, Pongal] -> Idli from Tiffin Box (30), Pongal from Dosa Point (50); total 80
order 2 [Idli, Vada] -> Idli from Tiffin Box (30), Vada from Udupi Corner (35); total 65
room left: Udupi Corner 2 of 3, Dosa Point 3 of 4, Tiffin Box 0 of 2
order 3 [Idli] -> Idli from Udupi Corner (40); total 40
dispatch order 1 -> true | the same message again -> false
  room left: Udupi Corner 1 of 3, Dosa Point 4 of 4, Tiffin Box 1 of 2
Tiffin Box raises Idli to 65; order 4 [Idli] -> Idli from Udupi Corner (40); total 40
  order 1, placed before the change, still totals 80
order 5 [Vada] -> Vada from Tiffin Box (50); total 50 | [Vada] again -> NO_ROOM
[Biryani] -> UNKNOWN_ITEM | [Idli, Idli] -> BAD_INPUT | dispatch 99 -> UNKNOWN_ORDER
add Dosa Point again -> DUPLICATE | items served: Udupi Corner 0, Dosa Point 1, Tiffin Box 1
Corner Cafe (Idli 30) sends a new menu, Vada 40 and Idli 0 -> BAD_INPUT
  then [Vada] -> UNKNOWN_ITEM | [Idli] -> Idli from Corner Cafe (30); total 30
added later, FoodOrdering unchanged, order [Idli, Vada]:
  OneRestaurant -> Idli from Udupi Corner (40), Vada from Udupi Corner (35); total 75
  BestRated -> Idli from Dosa Point (45), Vada from Udupi Corner (35); total 80
Tiffin Box (Idli 30, Vada 30, room 1), Udupi Corner (Idli 40, room 2), order [Idli, Vada]:
  CheapestPerItem -> NO_ROOM
  FullSearch -> Idli from Udupi Corner (40), Vada from Tiffin Box (30); total 70
CheapestPerItem vs a full search, 100,000 random orders of 1 to 4 items, 2 to 4 restaurants:
  servable 46,532 | greedy refused 3,068 of them | dearer than the best split 1,911 | invalid 0
8 threads x 50,000 calls on 5 restaurants (place 60%, dispatch 37%, new menu 3%):
  room outside 0..capacity in reads meanwhile 0 | other refusals 0 | order ids used twice 0
  each order freed its room once: true | then room == capacity: true | served == accepted: true
200,000 random calls, good and bad input -> FoodException 161,495 | any other exception 0
  BAD_INPUT 77,061, DUPLICATE 13,083, UNKNOWN_RESTAURANT 22,095,
  UNKNOWN_ITEM 4,955, UNKNOWN_ORDER 30,052, NO_ROOM 14,249
  refused calls that changed room or served counts: 0 | order ids consecutive: true
a buggy strategy, 2 items to a kitchen with room 1 -> IllegalStateException | room still 1
a strategy that casts its view to Restaurant to take room -> ClassCastException | room still 2
capacity and 3 prices at Integer.MAX_VALUE -> total 6,442,450,941 | room left 2,147,483,644
```

Real output of Q01_FoodOrderingTest.java (code/04-interview-questions/flipkart/) on JDK 21; JDK 11 and 25 are identical, three runs each. Random orders, Random(1): 2 to 4 restaurants, each selling each of 5 items with a chance of 6 in 10 at 10 to 100 rupees, room for 1 to 3 items with 0 to all taken; orders of 1 to 4 items. Threads: Random(10) to Random(17), menus from Random(2); only facts that hold for any interleaving are printed. Bad input, Random(7): bad names, menus, capacities, items and ids.

WHAT THEY ASK NEXT

L2 Which design patterns did you use, and why?

Strategy, for choosing restaurants: the rule changes, the service and its rules don't. No Singleton: each test builds its own. Factory if strategies are picked by name; Observer if others react to a dispatch.

L2 Add a new selection rule now, without rewriting the code.

New classes in the test, plugged into the constructor of an unchanged FoodOrdering: OneRestaurant (all from the cheapest restaurant with every item) sent [Idli, Vada] to Udupi Corner for 75; BestRated (ratings passed in), to Dosa Point and Udupi Corner for 80. Rules see read-only views and the service checks every pick, so no rule can overfill a kitchen.

L2 How do you handle errors and bad input?

One unchecked FoodException with a Reason enum, caught in one place by the driver; code switches on the reason, not the message. Everything is checked before the first change: 200,000 random calls threw nothing else, and no refusal changed room or counts. A strategy breaking the rules is a bug: IllegalStateException.

L3 Make it thread-safe: two customers want a restaurant's last free slot.

All of FoodOrdering's state is reached through its synchronized methods. By the Java spec one thread at a time holds the monitor (JLS 17.1), and releasing it happens-before the next lock (17.4.5), so choosing and taking room is one step that sees every earlier order: two orders can't both get the last slot. Orders are immutable (17.5), so callers need no lock. The 8-thread run is support, not proof. Per-restaurant locks in a fixed order add parallelism (Goldman Sachs Q33); Walmart Q1 has the general rules.

VERSION NOTE

> FoodOrdering compiles for Java 8; JDK 11, 21 and 25 gave identical output. Not run: cancelling, delivery partners and a timestamped command driver (parts of the problem in some reports), and ratings from a real service.

SEE ALSO

Ch 15 · Q13 The Strategy design pattern

Ch 4 · Q7 Creating a custom exception

The full Flipkart file

9 questions with full answers and tested code, organised by interview round and by experience level, plus 49 more asked questions on its More page.

Built for my Java Backend Developer Guide.

lnkd.in/dTvYVutD

The link is in the post.